Name : Manaswi Kabadi
Class : D15C
Roll no. : 27
Batch : B

EXP - 6

Aim : Apply K-Means and Hierarchical Clustering on sample datasets

DatasetSource : https://www.kaggle.com/datasets/shwetabh123/mall-customers

Dataset Description :
The Mall Customers dataset contains information about customers visiting a shopping mall, including their demographic details and spending behavior. The dataset consists of approximately 200 records and is widely used for customer segmentation tasks.
It includes attributes such as age, annual income, and spending score, which help in identifying patterns in customer purchasing behavior. Since the dataset does not contain any predefined labels, it is ideal for unsupervised learning algorithms like K-Means and Hierarchical Clustering.

Dataset Attributes:

Column Name	Data Type	Description
CustomerID	Numerical (Integer)	Unique identifier assigned to each customer
Gender	Categorical (String)	Gender of the customer (Male/Female)
Age	Numerical (Integer)	Age of the customer
Annual Income (k$)	Numerical (Float)	Annual income of the customer in thousands
Spending Score (1-100)	Numerical (Integer)	Score assigned by mall based on spending behavior

Target Variables Used in the Experiment
Since clustering is an unsupervised learning technique, there is no target variable used in this experiment. The objective is to group customers based on similarity in income and spending behavior.

K Means Clustering

Theory

K-Means clustering is a widely used unsupervised machine learning algorithm designed to partition a dataset into a predefined number of clusters, denoted by K. The primary objective of K-Means is to group data points such that points within the same cluster are highly similar to each other, while points belonging to different clusters are as dissimilar as possible.

The algorithm operates by minimizing the within-cluster sum of squares (WCSS), which measures the compactness of clusters. It begins by initializing K centroids randomly within the data space. Each data point is then assigned to the nearest centroid based on Euclidean distance. After assignment, the centroids are recalculated as the mean of all data points belonging to each cluster.

This iterative process continues until convergence is achieved, i.e., when cluster assignments no longer change significantly or the change in centroid positions becomes negligible. K-Means is computationally efficient and works best with numerical data and well-separated clusters.

In the context of the Mall Customers dataset, K-Means is used to segment customers based on their annual income and spending score, enabling the identification of distinct customer groups for targeted marketing strategies.

Mathematical Formulation of the Algorithm :

Let the dataset be X={x1,x2,…,xn}, with each xi∈Rd. The aim of K-Means is to partition the data into K clusters C1,C2,…,CK.

Objective function (WCSS):

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

where μk is the centroid of cluster Ck:

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

Algorithm Limitations :

- **Requirement to Predefine No. of Clusters :** K-Means requires the no. of clusters (K) to be specified before training. In datasets, the true no. of clusters is often unknown, which

can lead to incorrect grouping if K is poorly chosen.Condition of limitation: Performance degrades when the chosen K does not match the actual data distribution.

- **Sensitivity to Initial Centroid Selection :** The algorithm initializes centroids randomly, and different initializations can lead to different clustering results. This may cause the algorithm to converge to a local minimum rather than the global optimum.Condition of limitation: Results may vary significantly depending on initial centroid placement.
- **Assumption of Spherical Clusters :** K-Means assumes that clusters are spherical and evenly distributed. It relies on Euclidean distance, which makes it unsuitable for datasets with irregular or non-convex cluster shapes.Condition of limitation: Performs poorly when clusters have complex shapes or varying densities.
- **Sensitivity to Outliers and Noise :** Outliers can significantly affect centroid positions because K-Means uses mean values to compute centroids. This can distort cluster formation and reduce accuracy.Condition of limitation: Clustering results become unreliable in the presence of extreme values or noisy data.
- **Dependence on Feature Scaling :** K-Means relies on distance calculations, making it highly sensitive to the scale of input features. Features with larger values can dominate the clustering process. Condition of limitation: Without proper normalization or standardization, clustering may be biased toward certain features.

Methodology / Workflow :

Data Collection:

The Mall Customers dataset is loaded into a Pandas DataFrame. It contains approximately 200 customer records with demographic and behavioral attributes.

Data Cleaning and Preparation:

The dataset is checked for missing values and inconsistencies. Since the dataset is clean, minimal preprocessing is required.

Feature Selection:

Relevant numerical features such as Annual Income and Spending Score are selected for clustering, as they directly represent customer purchasing behavior.

Feature Scaling:

StandardScaler is applied to normalize the features so that both variables contribute equally to distance calculations.

Model Selection (K-Means):

K-Means clustering is chosen to group customers into distinct segments based on similarity in income and spending patterns.

Hyperparameter Tuning:

The optimal number of clusters (K) is determined using silhouette score and Elbow Method by evaluating values from 2 to 10.

Final Model Training and Cluster Assignment:

The model is trained using the optimal K, and cluster labels are assigned to each customer.

Cluster Profiling:

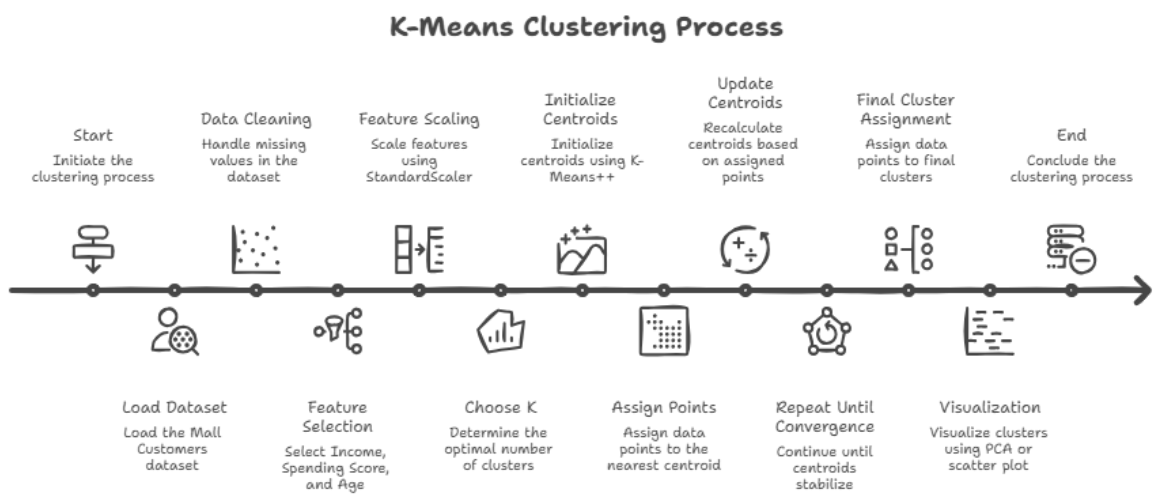
Cluster-wise averages of income and spending score are calculated to interpret customer segments.

Visualization:

Clusters are visualized using a 2D scatter plot, clearly showing separation between different customer groups.

Model Validation:

The Elbow Method is used to validate the optimal number of clusters by analyzing WCSS.



Code :

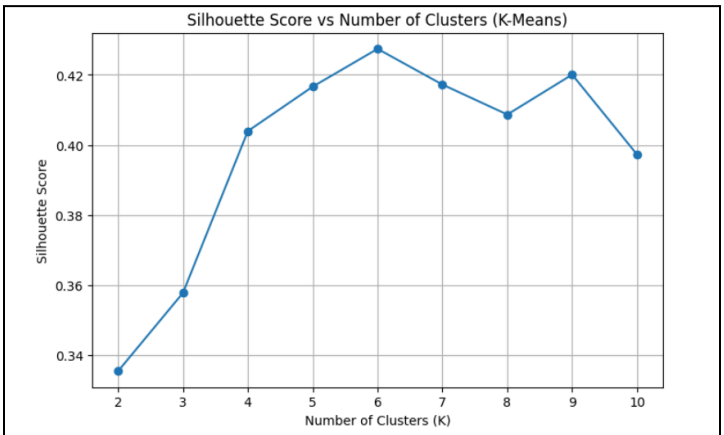
```
import pandas as pd
import numpy as np
from sklearn.preprocessing import
StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import
silhouette_score
from sklearn.decomposition import
PCA
import matplotlib.pyplot as plt
import seaborn as sns
# 1. Data Collection
df =
pd.read_csv("Mall_Customers.csv")
print(df.head())
# 2. Data Cleaning & Preparation
print(df.isnull().sum())
df = df.dropna()
```

```python
# 3. Feature Selection
features = ["Age", "Annual Income (k$)", "Spending Score (1-100)"]
X = df[features]
# 4. Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 5 & 7. Hyperparameter tuning:
# choose K using silhouette score
silhouette_scores = []
k_values = range(2, 11)

for k in k_values:
    kmeans = KMeans(
        n_clusters=k,
        init="k-means++",
        n_init=20,
        max_iter=300,
        random_state=42
    )
    labels = kmeans.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)

silhouette_scores.append(score)

plt.figure(figsize=(8, 5))
plt.plot(k_values, silhouette_scores, marker="o")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Silhouette Score")
plt.title("Silhouette Score vs Number of Clusters (K-Means)")
plt.grid(True)
plt.show()

best_k = k_values[np.argmax(silhouette_scores)]
print("Optimal number of clusters (K-Means):", best_k)

# 7. Final model training
kmeans_final = KMeans(
    n_clusters=best_k,
    init="k-means++",
    n_init=30,
    max_iter=300,
    random_state=42
)
df["KMeansCluster"] = kmeans_final.fit_predict(X_scaled)

# 8. Cluster Profiling
cluster_summary = df.groupby("KMeansCluster")[features].mean()
print("K-Means Cluster Summary:")
print(cluster_summary)

# 9. PCA visualization
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(8, 6))
plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=df["KMeansCluster"],
    cmap="viridis",
    s=30
)
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
```

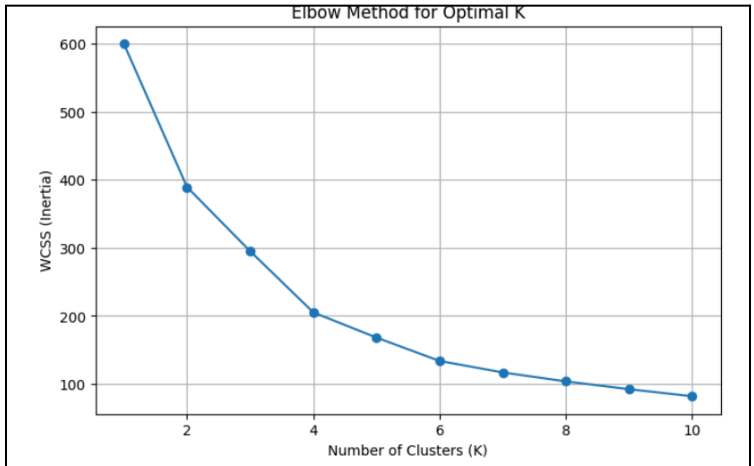
```
plt.title("Customer Segments using
K-Means (PCA 2D)")
plt.colorbar(label="Cluster")
plt.show()

cluster_counts =
df["KMeansCluster"].value_counts()
.sort_index()
plt.figure(figsize=(7, 5))
cluster_counts.plot(kind="bar")
plt.xlabel("Cluster Label
(K-Means)")
plt.ylabel("Number of Customers")
```

```
plt.title("Cluster Size
Distribution (K-Means)")
plt.grid(axis="y")
plt.show()

plt.figure(figsize=(8, 5))
sns.heatmap(cluster_summary,
cmap="viridis", annot=True,
fmt=".2f")
plt.title("Heatmap of Cluster
Feature Means (K-Means)")
plt.show()
```

OUTPUT :

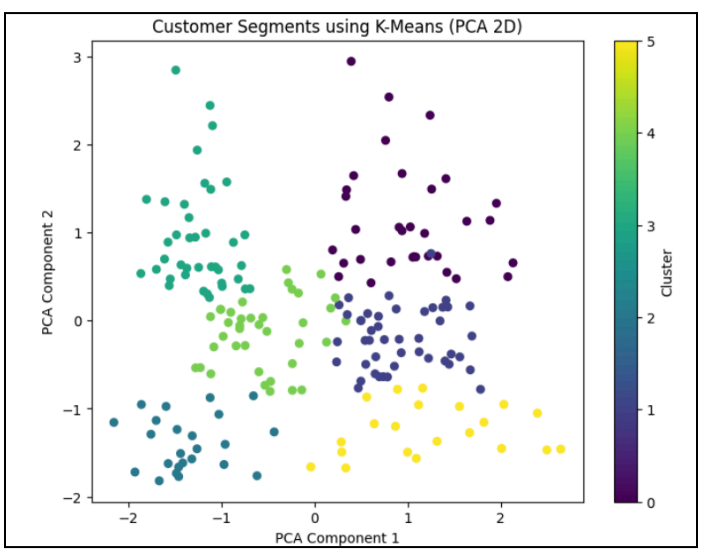


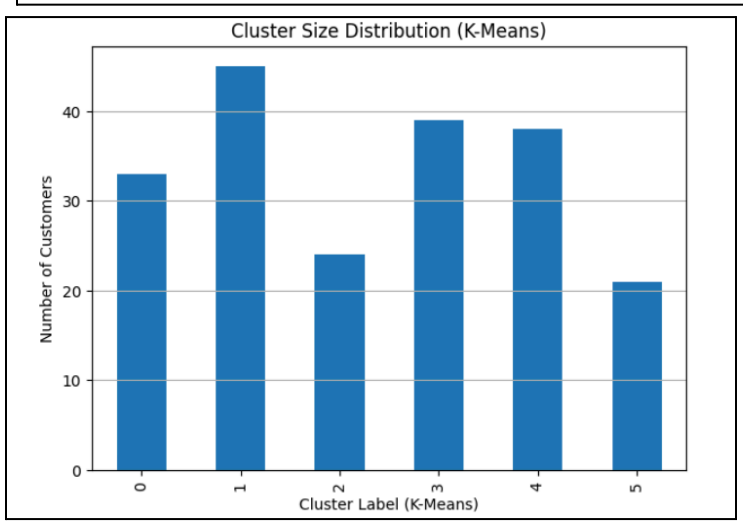

```
Optimal number of clusters (K-Means): 6
```

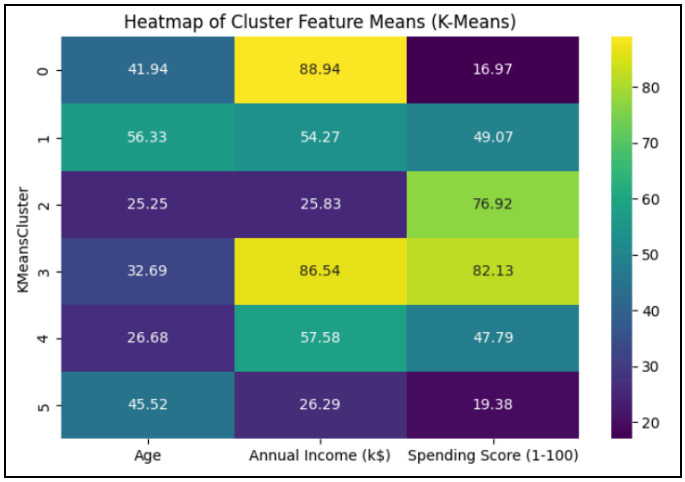



```
K-Means Cluster Summary:
```

KMeansCluster	Age	Annual Income (k$)	Spending Score (1-100)
0	41.939394	88.939394	16.969697
1	56.333333	54.266667	49.066667
2	25.250000	25.833333	76.916667
3	32.692308	86.538462	82.128205
4	26.684211	57.578947	47.789474
5	45.523810	26.285714	19.380952







Performance Analysis :

The performance of the K-Means clustering model on the Mall Customers dataset is evaluated using silhouette score, Elbow Method, PCA visualization, cluster distribution, and feature-based cluster profiling.

1. Optimal Number of Clusters Selection:

The silhouette score increases steadily and reaches its maximum value of approximately 0.43 at K = 6, indicating strong cluster separation. After K = 6, the score slightly decreases, suggesting that additional clusters do not improve clustering quality.
The Elbow Method also shows a noticeable bend around K = 5 to K = 6, where the rate of decrease in WCSS slows down significantly. This supports the selection of 6 clusters as the optimal number.

2. Cluster Visualization (PCA Analysis):

The PCA scatter plot shows clearly separated clusters with minimal overlap. Each cluster occupies a distinct region in the feature space, indicating strong inter-cluster separation and good clustering performance.

3. Cluster Size Distribution:

The cluster distribution graph shows that customers are fairly evenly distributed across all clusters. No cluster is extremely small or overly dominant, indicating a balanced and stable clustering solution.

4. Cluster Profiling (Heatmap Analysis):

The heatmap reveals clear differences in customer behavior across clusters:

- Some clusters represent high-income but low-spending customers
- Others show low-income but high-spending customers

- A few clusters represent high-income high-spending customers (premium segment)
- Remaining clusters show moderate or average behavior

This indicates that the model successfully identifies meaningful customer segments based on income and spending patterns.

Hyperparameter-Tuned Performance

After hyperparameter tuning using silhouette score evaluation, the optimal number of clusters was found to be K = 6, with the highest silhouette score of approximately 0.43.
The Elbow Method also supported this result, showing a clear bend around K = 5 to K = 6, indicating that increasing the number of clusters beyond this point provides minimal improvement in cluster compactness.
The final K-Means model trained with K = 6 produced well-separated clusters, as observed in the PCA visualization. The cluster distribution remained balanced, and the cluster profiles showed distinct differences in income and spending behavior.
This outcome suggests that the selected hyperparameters effectively capture the underlying structure of the dataset, and further increase in the number of clusters does not significantly improve clustering performance.

Hierarchical clustering

Theory :

Hierarchical clustering is an unsupervised machine learning algorithm that builds a hierarchy of clusters using a bottom-up (agglomerative) approach. Unlike K-Means, it does not require the number of clusters to be specified in advance. Instead, it creates a tree-like structure called a dendrogram, which visually represents how clusters are formed at different levels of similarity.
In agglomerative clustering, each data point initially starts as an individual cluster. The algorithm then repeatedly merges the two closest clusters based on a distance metric until all points are grouped into a single cluster. The merging process is guided by a linkage criterion such as single, complete, average, or Ward linkage.
Ward linkage is commonly used because it minimizes the increase in within-cluster variance, resulting in compact and well-separated clusters. The dendrogram can be analyzed to determine the optimal number of clusters by identifying large vertical gaps between merges.
In the Mall Customers dataset, hierarchical clustering helps in understanding the natural grouping of customers based on income and spending behavior, while also providing a visual interpretation of cluster relationships.

Mathematical Formulation of the Algorithm :

Given data X={x1,x2,…,xn}, hierarchical agglomerative clustering builds a hierarchy of clusters by successively merging the closest clusters until all points are in a single cluster.

● Initially, each data point is its own cluster.

● At each step, two clusters A and B that are most similar are merged according to a linkage criterion and a distance metric.

Distance metric (Euclidean):

$$d(x_i, x_j) = \sqrt{\sum_{l=1}^{d} (x_{il} - x_{jl})^2}$$

Ward linkage: At each step, merge the pair of clusters that results in the minimum increase in total within-cluster variance. This tends to produce compact, roughly spherical clusters.

The algorithm produces a dendrogram (tree), showing cluster merges at different distances. Cutting the dendrogram at a certain height corresponds to choosing a particular number of clusters K. Silhouette score (same as in K-Means experiment) is used to evaluate cluster quality for different choices of K.

Algorithm Limitations :

- **High computational and memory cost :** Time complexity is typically O(n2logn) or worse, and it often stores an n×n distance matrix, making it unsuitable for very large datasets.
- **Irreversible merges :** Once two clusters are merged they are never split again; early erroneous merges caused by noise or outliers cannot be corrected later.
- **Sensitivity to noise and outliers :** Outliers can form their own branches or force incorrect merges, which can distort the dendrogram and the resulting clusters.
- **Dependence on distance metric and linkage method :** Different choices of metrics (Euclidean, Manhattan, cosine) and linkages (single, complete, average, Ward) can lead to very different cluster structures and dendrograms.

Methodology / Workflow :

Data Collection: The Mall Customers dataset is loaded into a Pandas DataFrame, containing customer demographic and spending information.

Data Cleaning and Preparation: The dataset is checked for missing values and inconsistencies. Since the dataset is clean, minimal preprocessing is required.

Feature Selection: Relevant numerical features such as Annual Income and Spending Score are selected, as they represent customer purchasing behavior.

Feature Scaling: StandardScaler is applied to normalize the features, ensuring that all variables contribute equally to distance calculations.

Model Selection (Hierarchical Clustering): Agglomerative clustering is selected, where each data point starts as an individual cluster and clusters are merged iteratively.

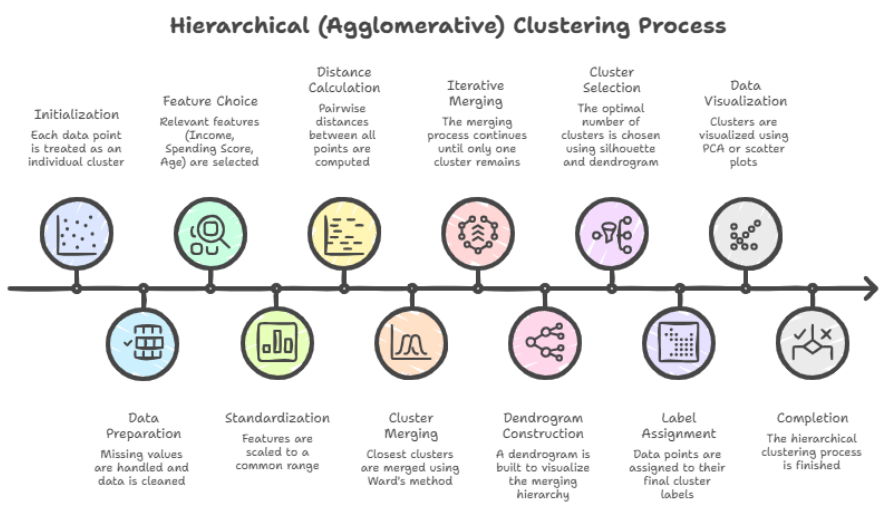
Hyperparameter Tuning: The optimal number of clusters is determined by evaluating silhouette scores for cluster values ranging from 2 to 10.

Final Model Training and Cluster Assignment: The hierarchical clustering model is trained using the optimal number of clusters, and cluster labels are assigned to each data point.

Cluster Profiling: Cluster-wise mean values are calculated to analyze differences in customer segments.

Visualization: Clusters are visualized using scatter plots to observe separation between groups.

Dendrogram Validation: A dendrogram is plotted using Ward linkage to visualize hierarchical relationships and determine the optimal number of clusters based on large vertical gaps.



Code :

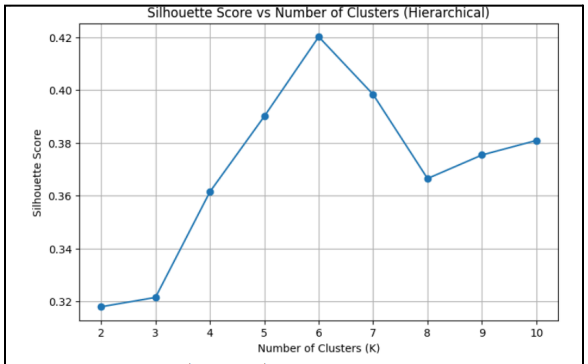
```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
from scipy.cluster.hierarchy import dendrogram, linkage

import matplotlib.pyplot as plt
import seaborn as sns
# 1. Data Collection
df = pd.read_csv("Mall_Customers.csv")
print(df.head())
# 2. Data Cleaning & Preparation
print(df.isnull().sum())
```

```
df = df.dropna()
# 3. Feature Selection
features = ["Age", "Annual Income (k$)", "Spending Score (1-100)"]
X = df[features]
# 4. Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# HYPERPARAMETER TUNING (Silhouette Score)
silhouette_scores = []
k_values = range(2, 11)

for k in k_values:
    model = AgglomerativeClustering(n_clusters=k, linkage="ward")
    labels = model.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    silhouette_scores.append(score)
# Plot
plt.figure(figsize=(8, 5))
plt.plot(k_values, silhouette_scores, marker="o")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Silhouette Score")
plt.title("Silhouette Score vs Number of Clusters (Hierarchical)")
plt.grid(True)
plt.show()
# Best K
best_k = k_values[np.argmax(silhouette_scores)]
print("Optimal number of clusters (Hierarchical):", best_k)
# ◆ FINAL MODEL
hierarchical_model = AgglomerativeClustering(n_clusters=best_k, linkage="ward")
df["HierCluster"] = hierarchical_model.fit_predict(X_scaled)
# ◆ CLUSTER PROFILING
cluster_summary = df.groupby("HierCluster")[features].mean()
print("\nHierarchical Cluster Summary:\n")
print(cluster_summary)
# ◆ PCA VISUALIZATION
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=df["HierCluster"], cmap="plasma", s=30)
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.title("Customer Segments using Hierarchical Clustering (PCA 2D)")
plt.colorbar(label="Cluster")
plt.show()
# DENDROGRAM
sample = X_scaled[:150]
linked = linkage(sample, method="ward")
plt.figure(figsize=(12, 6))
dendrogram(linked, truncate_mode="level", p=5)
plt.title("Hierarchical Clustering Dendrogram (Ward Linkage)")
plt.xlabel("Sample Index")
plt.ylabel("Distance")
plt.show()
# CLUSTER DISTRIBUTION
cluster_counts = df["HierCluster"].value_counts().sort_index()
plt.figure(figsize=(7, 5))
cluster_counts.plot(kind="bar")
```

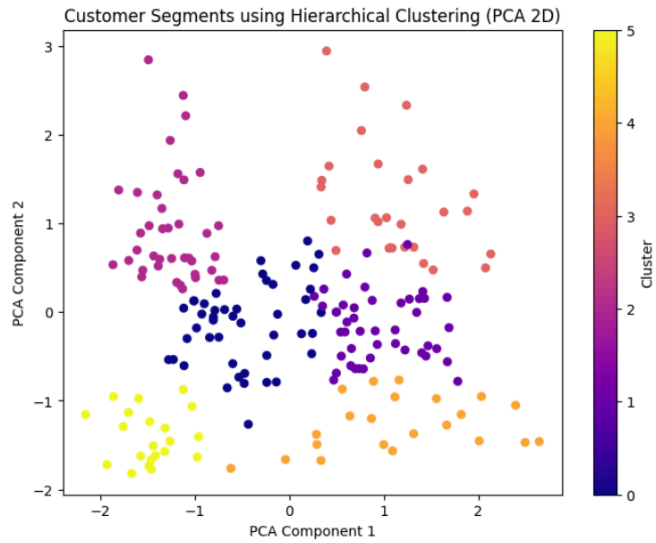
```
plt.xlabel("Cluster Label (Hierarchical)")
plt.ylabel("Number of Customers")
plt.title("Cluster Size Distribution (Hierarchical)")
plt.grid(axis="y")
plt.show()
```

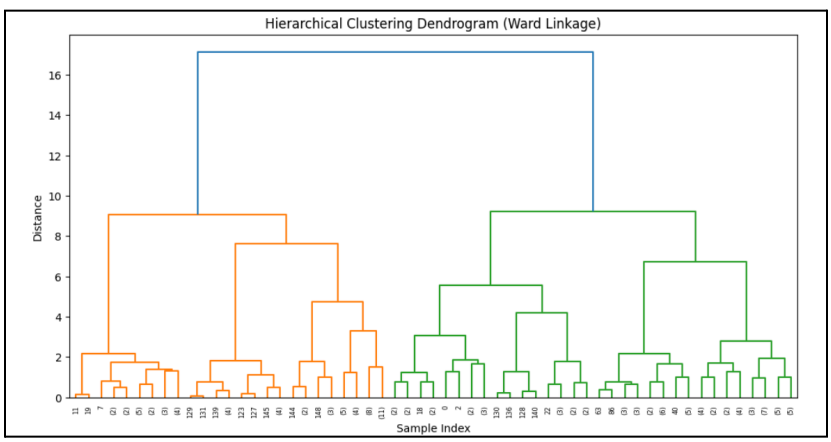
```
# HEATMAP
plt.figure(figsize=(8, 5))
sns.heatmap(cluster_summary, cmap="viridis", annot=True, fmt=".2f")
plt.title("Heatmap of Cluster Feature Means (Hierarchical)")
plt.show
```

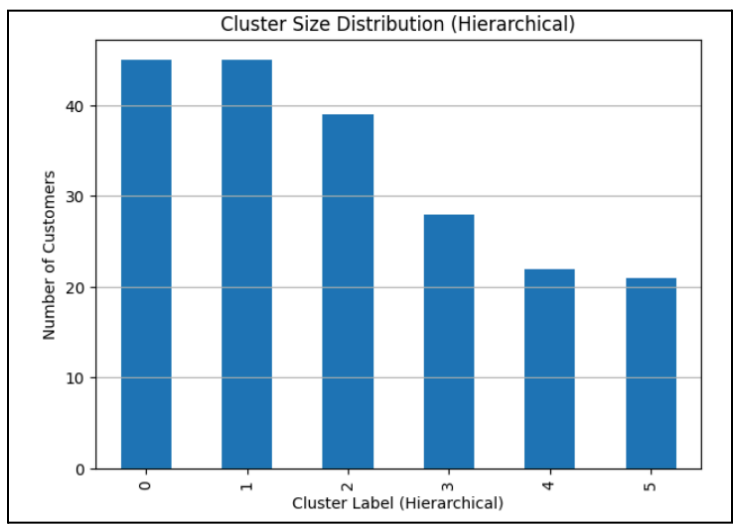
OUTPUT :

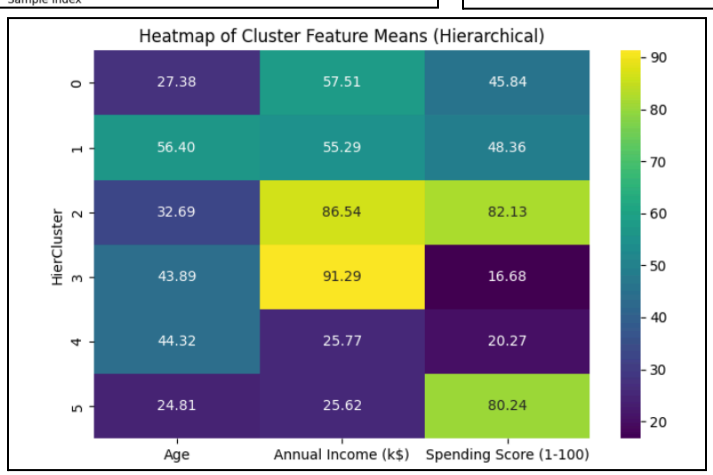



```
Optimal number of clusters (Hierarchical): 6

Hierarchical Cluster Summary:

                Age  Annual Income (k$)  Spending Score (1-100)
HierCluster
0         27.377778           57.511111               45.844444
1         56.400000           55.288889               48.355556
2         32.692308           86.538462               82.128205
3         43.892857           91.285714               16.678571
4         44.318182           25.772727               20.272727
5         24.809524           25.619048               80.238095
```









Performance Analysis :

The performance of the Hierarchical (Agglomerative) clustering model on the Mall Customers dataset is evaluated using silhouette score, dendrogram analysis, and cluster visualization.

1. Optimal Number of Clusters Selection:

The silhouette score reaches its maximum value of approximately 0.42 at K = 6, indicating strong cluster separation. After K = 6, the score decreases, suggesting that additional clusters do not improve clustering quality.

This confirms that 6 clusters provide the best balance between intra-cluster similarity and inter-cluster separation.

2. Dendrogram Analysis:

The dendrogram shows a significant vertical gap before the final merges, indicating a natural separation in the data. Cutting the dendrogram at this level results in approximately 6 distinct clusters, supporting the silhouette score result.

3. Cluster Visualization:

The clusters formed are well-separated and represent distinct customer groups based on income and spending behavior. The hierarchical method effectively captures the structure of the dataset.

Conclusion :

K-Means and Hierarchical clustering were successfully applied to segment mall customers based on age, annual income, and spending score. Both algorithms identified six meaningful customer groups with clear separation and distinct behavioral patterns, such as high-income high-spending and low-income low-spending segments. K-Means provided efficient and well-balanced clustering, while Hierarchical clustering offered additional insights through dendrogram visualization. The consistency in results from both methods confirms their effectiveness for customer segmentation and demonstrates their practical applicability in marketing and business decision-making.